1. Lab 1: Introduction to CML and Wireshark

2. Background

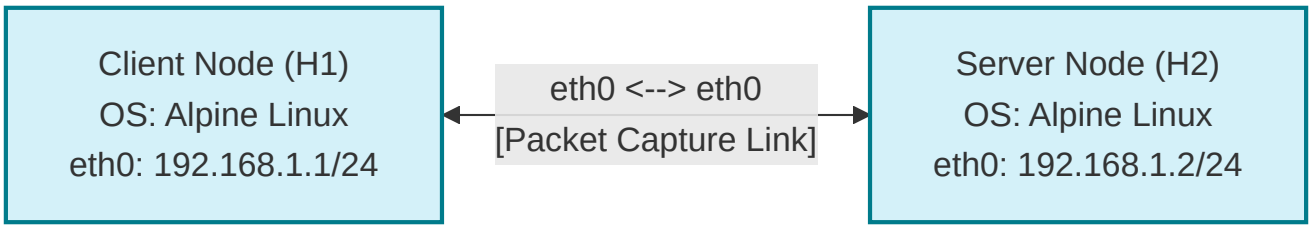
Cisco Modeling Labs (CML) is a network simulation platform that allows us to design, build, and test network topologies in a safe, virtualized environment. In this course, we will use CML to construct various network scenarios and observe how different protocols operate. Wireshark is a powerful network protocol analyzer that captures and displays the data traveling back and forth on a network in microscopic detail. By combining CML with Wireshark, you can actively generate traffic between simulated nodes and capture those packets to deeply understand network connectivity and protocol behaviors.

For more details on the features and components of CML, you can consult the official CML Quick Start Guide.

3. Objective / Purpose

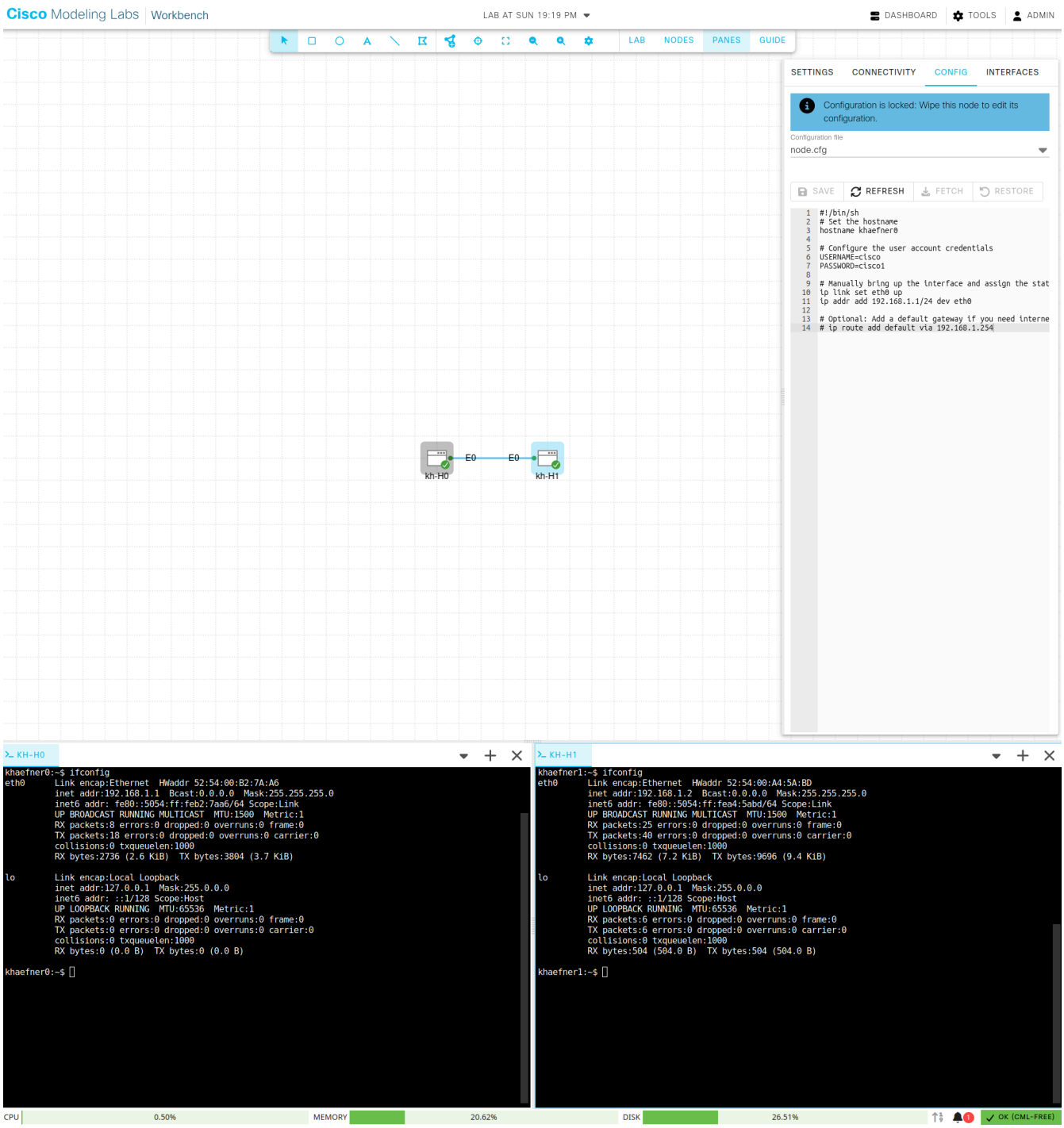
The objective of this lab is to introduce you to the basic operations of CML and Wireshark. You will deploy a simple network consisting of two Alpine Linux nodes, configure their hostnames and IP addresses using CML's boot configuration, and establish a link between them. Finally, you will test connectivity using the `ping` utility and capture the resulting ICMP traffic for analysis in Wireshark.

4. CML Topology and Configuration Procedure



Step 1: Deploying the Nodes

1. Log in to the CML UI and create a new lab.
2. From the node palette on the right, drag two **Alpine Linux** nodes onto the workspace.
3. Hover over one node, click and hold the blue link icon, and drag it to the second node to create a link between them.



Step 2: Configuring Hostnames and IP Addresses

You will configure the nodes so that their settings are applied automatically when they boot up. The naming convention for the nodes is `<initials>-H1` and `<initials>-H2`, where `<initials>` represents your first and last initials.

For Alpine Linux nodes in CML, the configuration in the **Edit Config** tab is executed as an autostart shell script when the node boots.

For this example, we will assume the initials `kh` and use the `192.168.1.0/24` subnet.

1. Click on the first Alpine Linux node in the workspace.
2. In the bottom pane, navigate to the **Node** tab and locate the **Hostname** field. Change it to `kh-H1` (adjust the initials to your own).
3. Still in the bottom pane, navigate to the **Edit Config** tab.
4. Enter the following autostart script to configure the hostname, user credentials, and interface settings:

```
#!/bin/sh
# Set the hostname
hostname kh-H1

# Configure the user account credentials
USERNAME=cs457
PASSWORD=cs457

# Manually bring up the interface and assign the static IP details
ip link set eth0 up
ip addr add 192.168.1.1/24 dev eth0

# Optional: Add a default gateway if you need internet/external routing
# ip route add default via 192.168.1.254
```

5. Click on the second Alpine Linux node.
6. In the **Node** tab, change the **Hostname** to `kh-H2`.
7. Navigate to its **Edit Config** tab and enter the script for the second host:

```
#!/bin/sh
# Set the hostname
hostname kh-H2

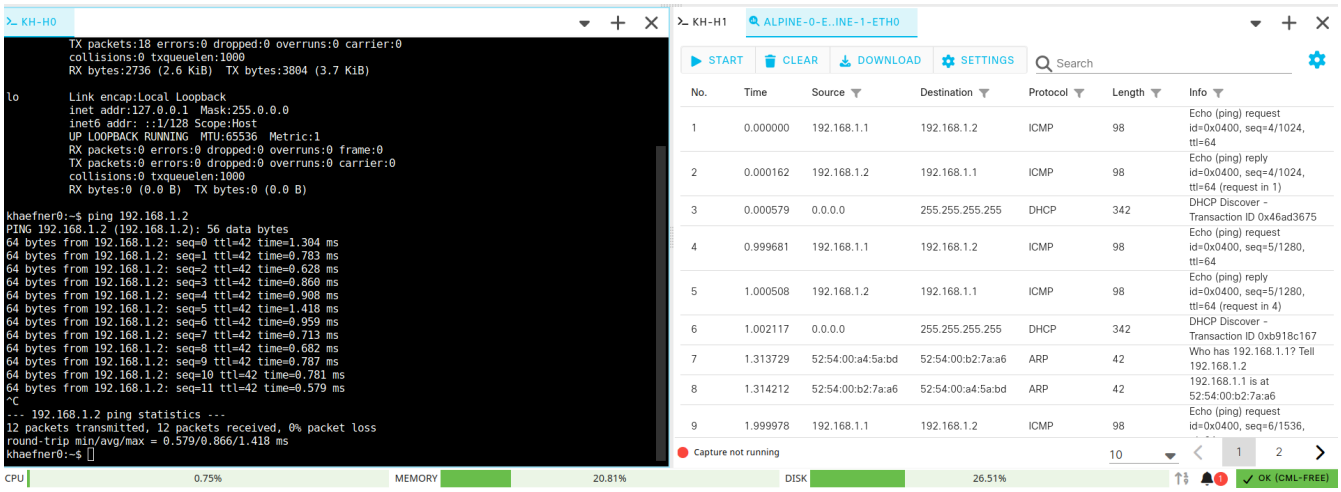
# Configure the user account credentials
USERNAME=cs457
PASSWORD=cs457

# Manually bring up the interface and assign the static IP details
ip link set eth0 up
ip addr add 192.168.1.2/24 dev eth0

# Optional: Add a default gateway if you need internet/external routing
# ip route add default via 192.168.1.254
```

Step 3: Starting the Nodes

1. Click the **Start Lab** button (the play icon) at the top of the workspace to boot up both nodes.
2. Wait a few moments for the nodes to fully boot. The nodes should turn green in the UI.



> ***NOTE - Node Deployment and Server Resources:*** *Starting a lab simulation deploys the virtual machine instances for the simulated nodes. Ensure the underlying CML server has sufficient CPU and RAM resources allocated. You can also start, stop, or wipe individual nodes inside a running lab by selecting the node and using the Workbench toolbar actions, rather than restarting the entire topology. For details, refer to the CML Node Actions Guide.*

5. CML Connectivity and Packet Capture

> *[!IMPORTANT]* ***REMINDER: All screenshots, console outputs, and packet captures (.pcap files) must be taken from the SAME active lab session. Mixing screenshots or trace files across different runs or sessions is not permitted.***

Step 1: Starting the Packet Capture

Before generating traffic, you need to start a packet capture on the link connecting the two nodes.

1. In the CML workspace, click on the link (the line) connecting your two Alpine Linux nodes.
2. In the bottom pane, navigate to the **Packet Capture** tab.
3. Click the **Start** button to begin capturing traffic on that link.

> ***TIP - Packet Capture Settings & Berkeley Packet Filters (BPF):*** *By default, CML captures up to 50 packets of any type and stops automatically. You can customize the capture limits and filters by selecting the link and clicking the* ***Settings*** *button in the* ***Packet Capture*** *pane:*
>
> - ***Max Packets****: The maximum number of packets to capture (default is 50).*
> - ***Max Time****: The duration limit (in seconds) for the capture.*
> - ***BPF****: An optional Berkeley Packet Filter expression to capture only specific traffic (e.g.,* `icmp` *,* `arp` *, or* `ip src 192.168.1.1` *).*
> - ***BPF Templates****: Predefined filter templates to help build your expression. Note: You must set a value for either Max Packets or Max Time. If both are set, the capture stops when the first limit is reached. For more info, check out Running a Packet Capture.*

> ***WARNING - Capture Expiration and Limits:***
>
> - *Only one packet capture is supported per link at a time. Starting a new capture on the same link will discard the previous session.*

> - *Packet captures are temporarily stored on the CML server and are **wiped** when you stop the link or the lab. Make sure to download the capture before stopping your topology.*

Step 2: Testing Connectivity

1. Click on the first node (`H1`) and navigate to the **Console** tab in the bottom pane. Press **Enter** to get a prompt.
2. Click on the second node (`H2`) and open its **Console** tab as well. Press **Enter** to get a prompt.

> ***NOTE - Console Access Methods in CML:*** *There are three primary ways to access the console of a running node:*
>
> 1. ***Panes Console (Browser UI)**: Direct access via the Workbench bottom pane (default method).*
> 2. ***Console Server**: Connect directly using telnet/SSH to CML's console gateway port.*
> 3. ***Breakout Tool**: A local utility that redirects console ports to local TCP ports, letting you use terminal emulators like PuTTY or SecureCRT. Refer to Connecting to a Node's Console for more details.*
> 4. *In the console of `H1` , ping `H2` by typing:*

```
ping -c 4 192.168.1.2
```

4. In the console of `H2` , ping `H1` by typing:

```
ping -c 4 192.168.1.1
```

Step 3: Downloading the Capture

1. Click back on the link connecting the two nodes.
2. Go to the **Packet Capture** tab and click **Stop**.
3. Click the **Download** button to save the capture file (a `.pcap` file) to your local machine.

> ***TIP**: The downloaded PCAP file is in standard packet capture format. Once saved, you can open and analyze it using Wireshark on your local machine. For additional steps, refer to Downloading a Packet Capture.*

6. Wireshark Overview and Getting Started

6.1 Packet Sniffer Structure

One’s understanding of network protocols can often be greatly deepened by “seeing protocols in action” and by “playing around with protocols” – observing the sequence of messages exchanged between two protocol entities, delving down into the details of protocol operation, and causing protocols to perform certain actions and then observing these actions and their consequences.

This can be done in simulated scenarios (like CML) or in a “real” network environment such as the Internet. In the Wireshark labs you’ll be doing in this course, you’ll observe the network protocols "in action," interacting and exchanging messages with other protocol entities. You’ll observe, and you’ll learn, by doing.

The basic tool for observing the messages exchanged between executing protocol entities is called a **packet sniffer**. As the name suggests, a packet sniffer captures (“sniffs”) messages being sent/received from/by your computer; it will also typically store and/or display the contents of the various protocol fields in these captured messages.

A packet sniffer itself is passive. It observes messages being sent and received by applications and protocols running on a network interface, but never sends packets itself. Similarly, received packets are never explicitly addressed to the packet sniffer. Instead, a packet sniffer receives a copy of packets that are sent/received.

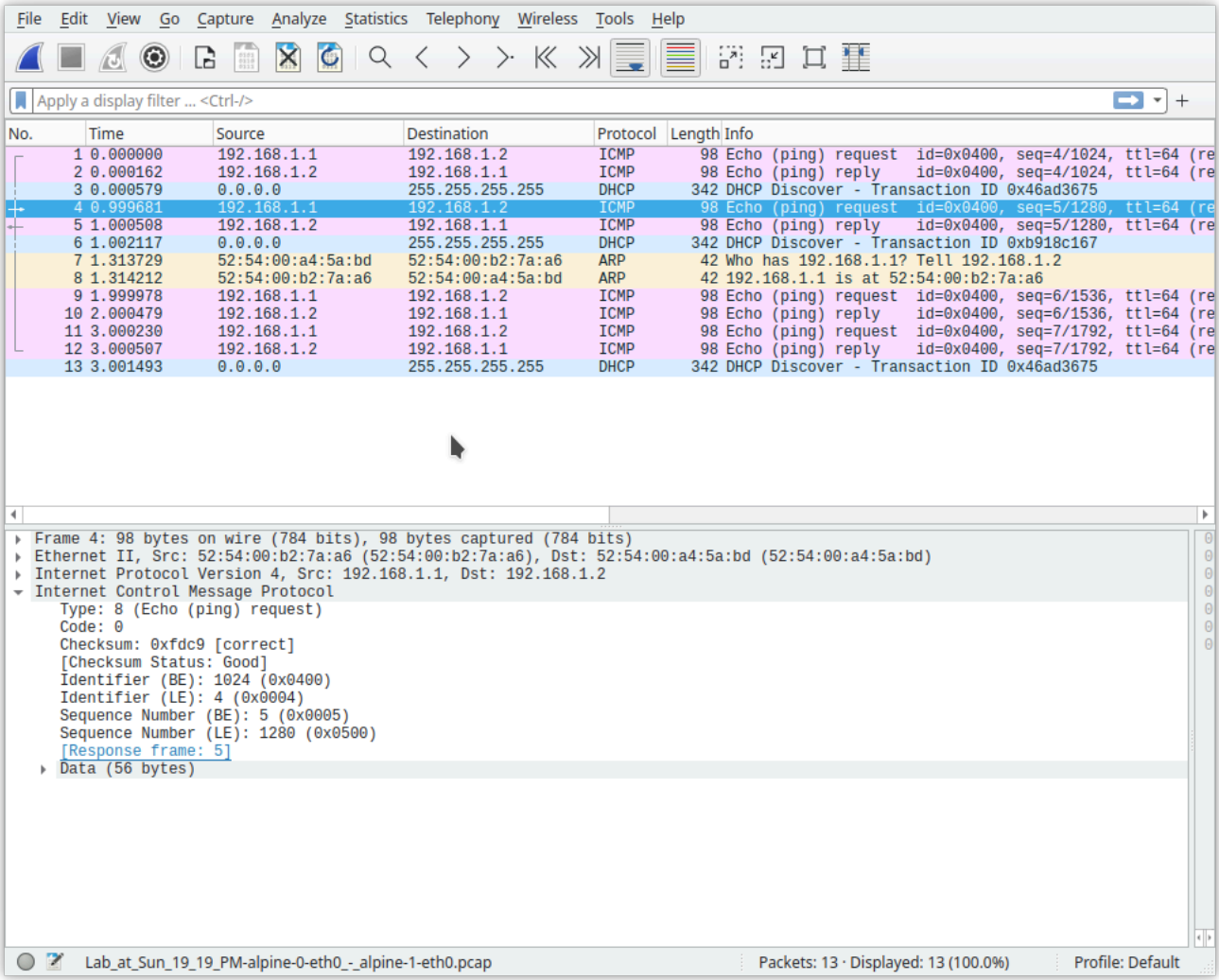


Figure 1 shows the structure of a packet sniffer. At the right of Figure 1 are the protocols (in this case, Internet protocols) and applications (such as a web browser or email client) that normally run on a computer. The packet sniffer, shown within the dashed rectangle, is an addition to the usual software in the computer, and consists of two parts:

1. **The packet capture library:** Receives a copy of every link-layer frame that is sent from or received by the computer over a given interface (link layer, such as Ethernet or WiFi). Messages exchanged by higher-layer protocols (such as HTTP, TCP, UDP, DNS, or IP) all are eventually encapsulated in link-layer frames that are transmitted over physical media. Capturing all link-layer frames thus gives you all messages sent/received across the monitored link.
2. **The packet analyzer:** Displays the contents of all fields within a protocol message. In order to do so, the packet analyzer must "understand" the structure of all messages exchanged by protocols. For example, the packet analyzer understands the format of Ethernet frames, and so can identify the IP datagram within an Ethernet frame. It also understands the IP datagram format, so that it can extract the TCP segment within the IP datagram. Finally, it understands the TCP segment structure to extract the higher-layer protocol content (such as HTTP or ICMP).

We will be using the Wireshark packet sniffer (http://www.wireshark.org/) for these labs, allowing us to display the contents of messages being sent/received at different levels of the protocol stack. Wireshark is a free, open-source

network protocol analyzer that runs on Windows, Mac, and Linux/Unix computers. It is stable, has a large user base, and features a well-designed user interface.

6.2 Getting Wireshark

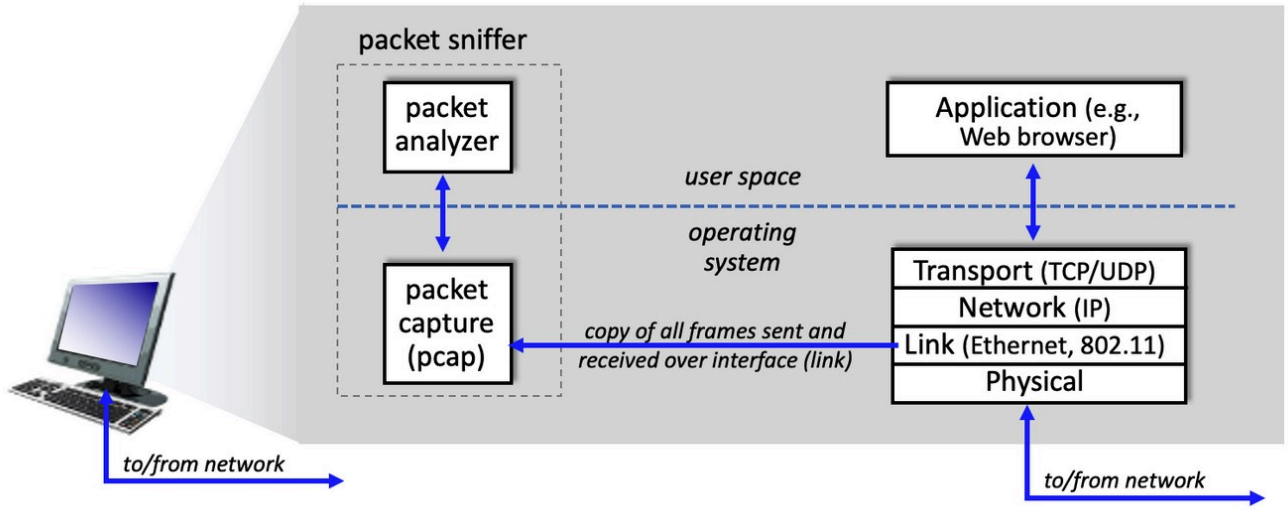
In order to run Wireshark, you'll need to have access to a computer that supports both Wireshark and the `libpcap` (Linux/Mac) or `Npcap` / `WinPcap` (Windows) packet capture library. The capture library is typically installed automatically when you install Wireshark.

Download and install the Wireshark software:

- Go to http://www.wireshark.org/download.html and download the appropriate installer for your operating system.

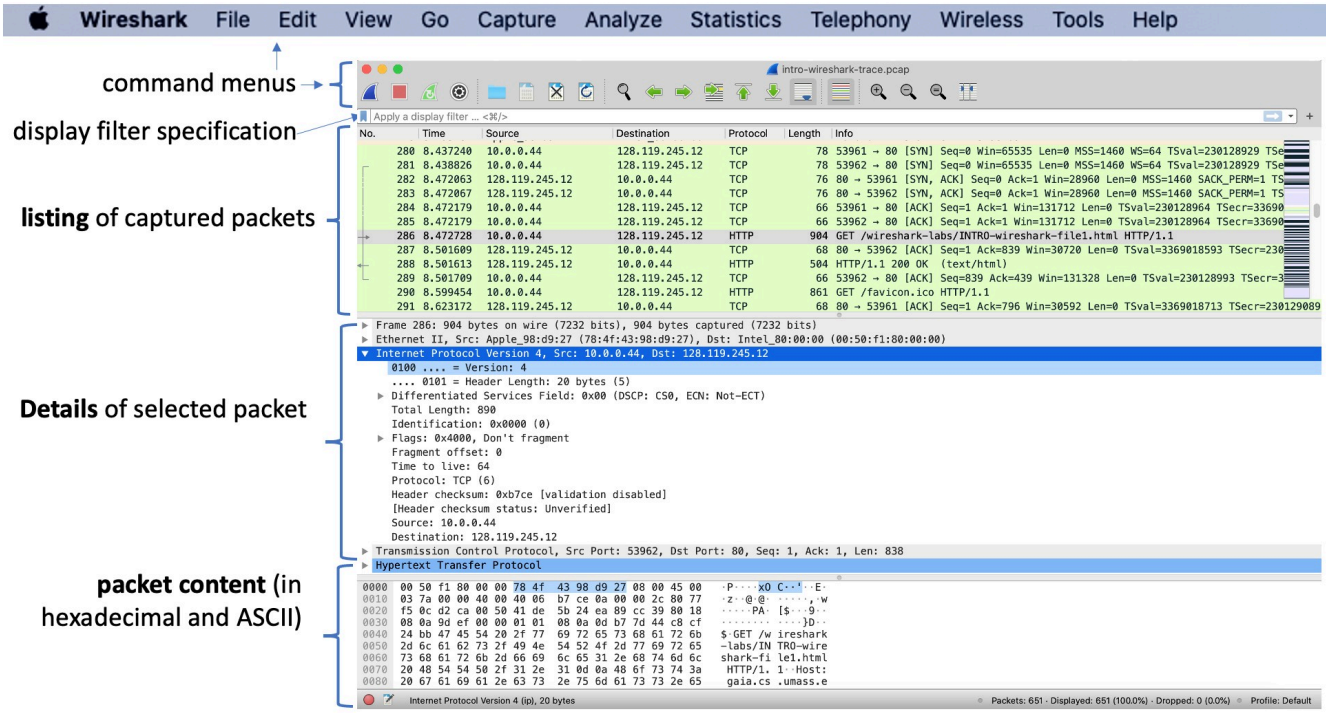
6.3 Running Wireshark

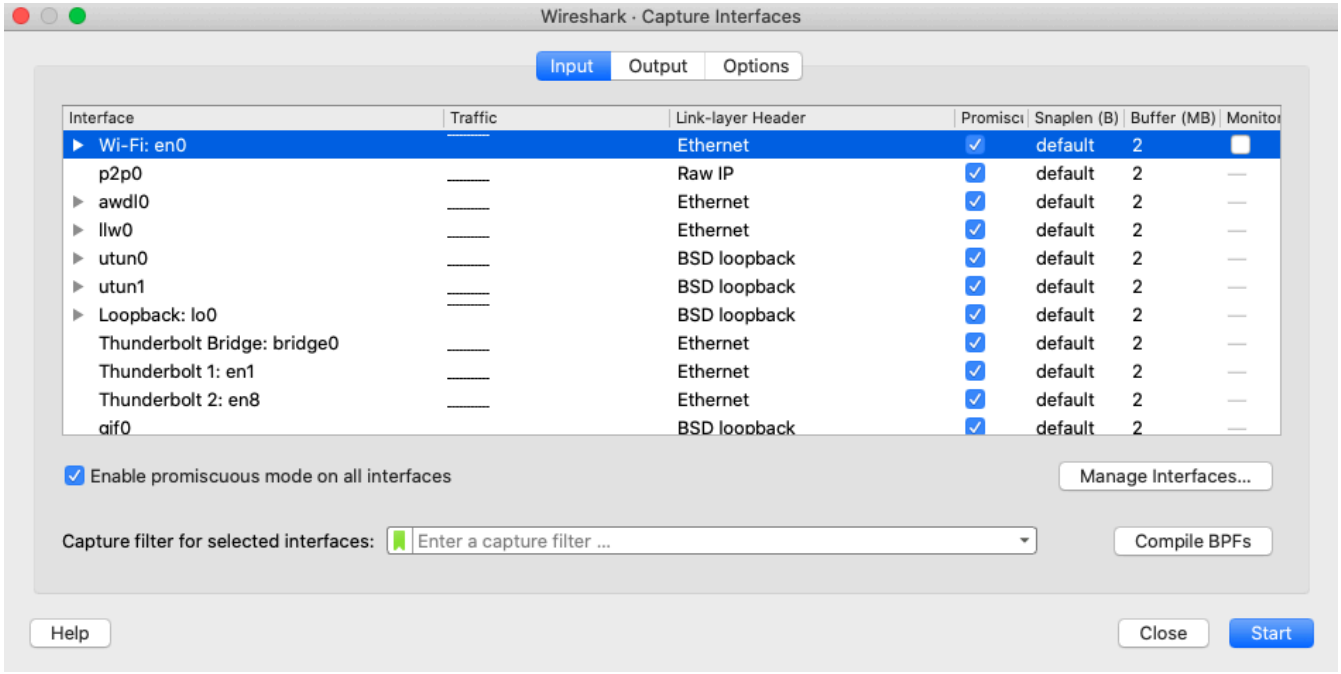
When you run the Wireshark program, you'll get a startup screen that looks similar to Figure 2. Note that different versions of Wireshark will have slightly different startup screens, but the core functionality remains identical.



Under the **Capture** section on the startup screen, there is a list of interfaces available on your machine. If you were doing a local packet capture, you would select the interface that currently has network activity (e.g., Wi-Fi or Ethernet) and double-click it (or click the blue fin icon) to start capturing live traffic.

For local captures, Figures 4a and 4b show the capture interface selection screens on Windows and macOS.





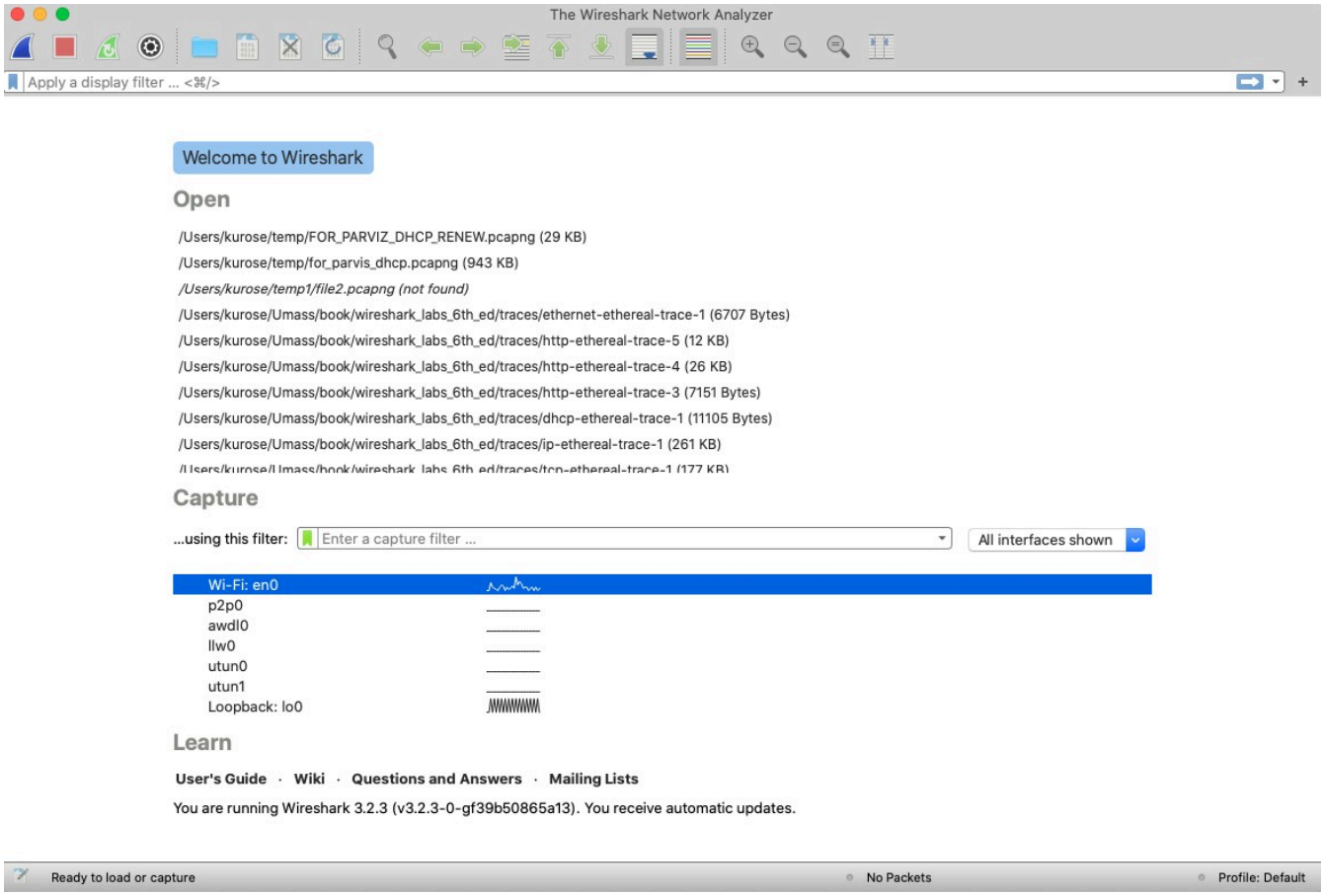
Note: For this lab, you do not need to perform a local capture on your machine's physical network adapter. Instead, we will be opening the `.pcap` capture file that you downloaded from your CML environment.

6.4 The Wireshark Interface

To load your capture file into Wireshark:

1. Open Wireshark.
2. Select **File -> Open** from the main menu.
3. Browse to and select the `.pcap` file you downloaded from CML in Section 5.

Once loaded, the Wireshark window should display the captured packets as shown in Figure 3.



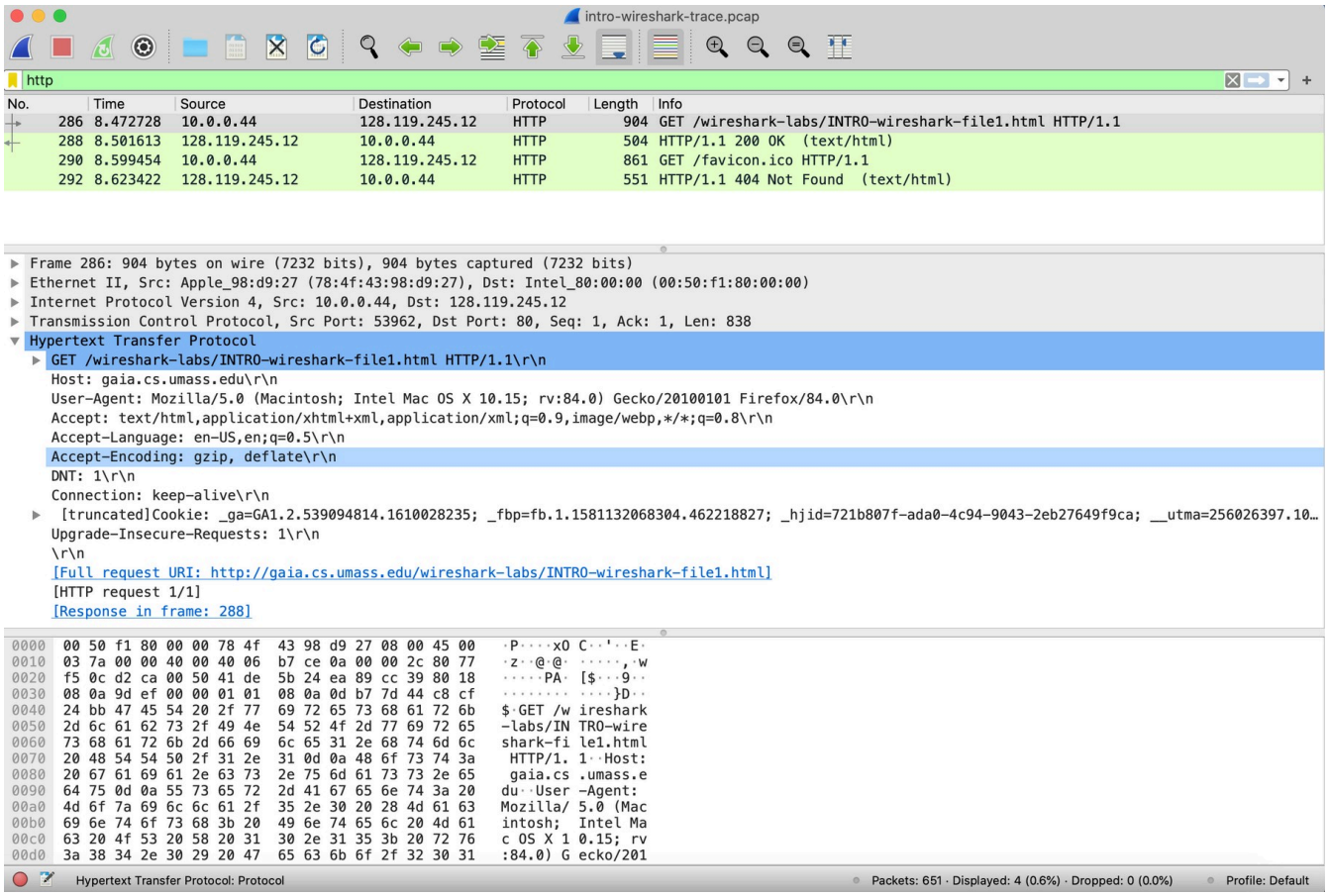
The Wireshark interface has five major components:

- **Command Menus:** Standard pull-down menus located at the top of the window (e.g., File, Edit, View, Go, Capture, Analyze, Statistics, Help). The **File** menu allows you to save captured packet data or open existing files.
- **Packet-Listing Window:** Displays a one-line summary for each packet captured. This includes the packet number (assigned by Wireshark for display purposes; it is not a packet header field), the capture time, source and destination IP addresses, protocol type, and protocol-specific information. The list can be sorted by clicking on any column header.
- **Packet-Header Details Window:** Provides protocol details about the packet currently selected in the packet-listing window. These details are grouped hierarchically (e.g., Frame, Ethernet II, Internet Protocol, and transport/application layers) and can be expanded or collapsed by clicking the arrowheads or plus/minus icons.
- **Packet-Contents Window:** Displays the entire raw contents of the captured frame in both hexadecimal and ASCII formats.
- **Packet Display Filter Field:** Located near the top of the window. You can type protocol names or filter expressions (e.g., `icmp`, `arp`, `ip.addr == 192.168.1.1`) to hide packets that do not match the criteria.

6.5 Analyzing Your CML Capture in Wireshark

Now let's take Wireshark for a test run using the CML capture file:

1. Open your CML `.pcap` file in Wireshark.
2. Observe the list of packets. You should see a mix of **ICMP** (from your ping tests) and potentially **ARP** packets.
3. Locate the display filter bar at the top, type `icmp` (in lowercase), and press **Enter** (or click Apply). You will notice that all other packets (like ARP) disappear, leaving only the ICMP Echo Requests and Replies.
4. Select the first ICMP Echo Request packet (sent from `192.168.1.1` to `192.168.1.2`).
5. In the **Packet-Header Details Window**, locate the entry for **Internet Control Message Protocol**. Expand this section to view the ICMP-specific fields.
6. Collapse the Frame, Ethernet, and Internet Protocol sections, and maximize the Internet Control Message Protocol section. Your layout should resemble the hierarchical organization shown in the example in Figure 5.



(Note: Figure 5 shows an example of detailed protocol expansion using HTTP. In your current capture, you will expand the ICMP header instead of HTTP.)

7. Observe the **Type** and **Code** fields inside the ICMP header. Note what you see for an Echo Request.
8. Select the corresponding ICMP Echo Reply packet (sent from `192.168.1.2` back to `192.168.1.1`). Observe its Type and Code fields.

7. Troubleshooting

Issue Encountered: [Describe what went wrong, if anything]
Discovery Method: [How did you identify the issue?]
Resolution: [How did you resolve it?]

AI Policy & Acceptable Use Guidelines

 AI Policy & Acceptable Use Guidelines - Yellow Icon

Artificial Intelligence tools can be powerful learning aids, but they must be used ethically and responsibly to ensure you develop core networking skills.

- **Lab Assignments:** Do not use AI to perform lab steps or write your lab submissions. However, if you get stuck on a concept or technical obstacle, you may use AI as a virtual tutor to ask clarifying questions and help you troubleshoot.

---

8. What to Hand In

Please submit the following:

- A screenshot of your CML topology showing the two connected nodes.
- Screenshots of the running consoles for both hosts (`H1` and `H2`), clearly showing the successful `ping` output and the customized hostnames.
- The Wireshark `.pcap` file containing the captured ICMP messages.
- A screenshot of your Wireshark window displaying the captured ICMP (ping) traffic, with the ICMP header details expanded in the packet-details pane.
- A document containing the answers to the questions in Section 8.

8. Questions

Note: If you are unable to run the CML lab to generate your own capture, your instructor may provide a pre-captured trace file (`intro-wireshark-trace1.pcap`) for you to load into Wireshark to answer these questions.

1. Which protocols are shown as appearing (i.e., are listed in the Wireshark “protocol” column) in your CML packet capture trace file?
2. How long did it take from when the first ICMP Echo Request message was sent until the corresponding ICMP Echo Reply was received? (By default, the value of the Time column in the packet-listing window is the elapsed time in seconds since Wireshark tracing began. To view it in time-of-day format, select **View -> Time Display Format -> Time-of-day**.)
3. What is the IP address of host H1? What is the IP address of host H2?
4. Expand the information on the ICMP message in the Wireshark “Details of selected packet” window. What are the values of the **Type** and **Code** fields for the ICMP Echo Request packet? What are the values for the ICMP Echo Reply packet?
5. Expand the Ethernet II header for the first ICMP Echo Request packet. What is the Source MAC address? What is the Destination MAC address? Does the Destination MAC address match the MAC address of host H2?
6. In your own words, explain the difference between a packet capture library (such as `libpcap`) and a packet analyzer (such as Wireshark) based on the Packet Sniffer structure description in Section 6.1.